SHBT 261 Mini Project 1
Victoria Alkin

1. Introduction

The goal of this project is to train and evaluate multiple methods for image classification on the Caltech-101 dataset, which has roughly 9000 images spanning 101 categories. The task is to build classifiers that can correctly assign images to one of the classes and to understand how performance differs between classical machine learning and deep learning methods, and how design choices affect performance.
I compared one classical machine learning baseline and three deep learning models. The classical method uses Histogram of Oriented Gradients (HOG) features paired with a linear Support Vector Machine (SVM) classifier (LinearSVC). The deep learning methods are a Simple CNN trained from scratch and two transfer learning methods, Residual Network 18 (ResNet-18) and EfficientNet-B0, each initialized from ImageNet-pretrained weights and fine-tuned on Caltech-101. These methods are systematically compared using multiple evaluation metrics including overall accuracy, per-class accuracy, confusion matrices, and macro and weighted average precision/recall/F1. Finally, I ran ablation studies, including an experiment that tests the effect of image size and an experiment that tests the effect of data augmentation.

2. Methods

2.1 Dataset and Split
The Caltech-101 dataset was downloaded from Kaggle. The downloaded archive contained 102 folders rather than the 101 object categories used for classification. The additional folder, BACKGROUND_Google, contains background images rather than an object class, and was therefore excluded from the label set. After removing this folder, the dataset comprised the expected 101 classes. I constructed a stratified train, validation, and test split with proportions of 70%, 15%, and 15%, respectively, ensuring that each split preserved the overall class distribution. Class sizes were notably imbalanced (some categories containing substantially more images than others), motivating the use of stratification and the reporting of both overall and class-balanced metrics.

2.2 Preprocessing for all Primary Models
To ensure a fair comparison across all methods, including classical and deep learning methods, I applied the same base preprocessing for all primary models. Each image was converted to RGB and resized to a fixed 128×128 resolution using direct resizing. All deep learning models used ImageNet preprocessing, where images were converted to RGB, resized to the target resolution, and normalized channel-wise using the ImageNet mean and standard deviation (mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]). Any deviations were reserved for ablation studies.

2.3 Classical Machine Learning Baseline: Linear Support Vector Machine
Feature Extraction with Histogram of Oriented Gradients (HOG). For the classical baseline, I extracted Histogram of Oriented Gradients (HOG) features from the resized 128×128 RGB images. HOG was computed with 9 orientations, 8×8 pixels per cell, 2×2 cells per block, and L2-Hys block normalization. For color handling, HOG was computed in a channel-aware manner equivalent to computing HOG over RGB channels and concatenating, producing a fixed-length feature vector of 8100 dimensions per image. To improve efficiency and reproducibility, HOG features were precomputed for train, validation, and test sets and saved.

Linear Support Vector Machine (SVM). I trained a linear Support Vector Machine using scikit-learn's LinearSVC. The regularization parameter C was selected via a small grid search on the validation set ($C \in \{0.01, 0.1, 1, 10\}$). For each C, the model was trained on the training split and evaluated on the validation split using accuracy. The best-performing C value that maximized validation accuracy was selected. After selecting the best C (C=0.01), I retrained the model on the combined train and validation sets and evaluated on the test set.

2.4 Deep Learning Model 1: Convolutional Neural Network from Scratch
Model architecture. I trained a compact convolutional neural network from scratch consisting of four convolutional blocks with channel widths 32→64→128→256, each block using 3×3 convolution, batch normalization, ReLU, and 2×2 max pooling (spatial resolution reduced from 128×128 to 8×8). The classifier head flattened the final feature map and used a fully connected stack (256·8·8 → 512 → 101) with dropout for regularization.

Training procedure. The network was trained for 100 epochs using the Adam optimizer with a learning rate of 1×10^{-4} and a batch size of 32. After each epoch, I evaluated validation accuracy and saved the best checkpoint by validation accuracy. That best-validation checkpoint was then evaluated on the test set. This strategy prevents overfitting to the training set (by picking the best-validation checkpoint rather than the best-training or last checkpoint) while avoiding test-set leakage.

2.5 Deep Learning Model 2: ResNet-18 pretrained on ImageNet
Model architecture. I used a ResNet-18 convolutional neural network initialized with ImageNet-pretrained weights. The original 1000-class classification head was replaced with a new fully connected layer mapping to 101 classes.

Fine-tuning and training. I performed full fine-tuning, updating all network parameters (backbone + classification head) on Caltech-101. This allows the pretrained representation to adapt to the target dataset. The model was trained with cross-entropy loss using Adam (learning rate = 1e-4, batch size = 32) with 100 epochs. After each epoch, validation accuracy was computed and the best checkpoint by validation accuracy was saved. The best-validation checkpoint was evaluated once on the test set.

2.6 Deep Learning Model 3: EfficientNet-B0 pretrained on ImageNet
Model architecture. I used EfficientNet-B0 initialized with ImageNet-pretrained weights, replacing the classifier layer to output 101 classes.

Fine-tuning protocol and training. I performed full fine-tuning, updating all network parameters on Caltech-101. Training used cross-entropy loss and Adam optimizer (learning rate = 1e-4, batch size = 32) with 100 epochs. Validation accuracy was evaluated each epoch, and the best checkpoint by validation accuracy was saved and then evaluated on the test set.

As a note, I wanted to run additional models such as ViT but ran into limitations with Colab and drive storage.

2.7 Hardware and Software
All experiments were run in Google Colab using an NVIDIA Tesla T4 GPU. Deep learning models were implemented in PyTorch. For the classical baselines, feature extraction and modeling were implemented using scikit-image (Histogram of Oriented Gradients), scikit-learn (LinearSVC, SGDClassifier, and evaluation metrics), with NumPy/Pandas for data handling, Pillow (PIL) for image I/O, joblib for saving trained models, and matplotlib for plotting.

2.8 Evaluation Metrics
I report overall test accuracy, per-class accuracy, confusion matrices, and precision, recall, and F1 with both macro and weighted averaging. I also report top-5 accuracy, computed by checking whether the true class is among the top-5 classes.

2.9 Ablation Studies
To quantify the impact of key design choices, I performed two ablation studies on image size and data augmentation. To quantify the effect of input image size on classification performance, I conducted a controlled ablation comparing image sizes of 128×128 (baseline) and 64×64 (ablation) across all four models. To test the effect of data augmentation, training was repeated for the three deep learning models with random crop/flip augmentation enabled versus disabled. The data augmentation applied RandomResizedCrop (scale 0.8–1.0, ratio 0.9–1.1) and RandomHorizontalFlip (p=0.5). In both studies, all other design choices were kept constant, including RGB input, train/validation/test splits (seed = 42), optimizer (Adam), learning rate (1e-4), batch size (32), C-sweep procedure, feature extraction (all where applicable).

3. Results
3.1 Hyperparameter Selection for Classical Machine Learning Model
To determine the optimal regularization strength for the classical baseline, I performed a sweep over the inverse regularization parameter C for the Linear Support Vector Classifier trained on HOG features. The parameter C controls the trade-off between maximizing the margin and minimizing classification error. Smaller values of C impose stronger regularization, whereas larger values reduce regularization and allow the classifier to fit the training data more closely. Validation accuracy was evaluated across a logarithmic grid of $C \in \{10^{-2}, 10^{-1}, 10^{0}, 10^{1}\}$. As shown in Figure 1, the highest validation accuracy was achieved at $C = 0.01$. Performance decreased gradually as C increased, indicating that stronger regularization improved generalization on this dataset. The selected value $C = 0.01$ was therefore used for final training before evaluating on the held-out test set.

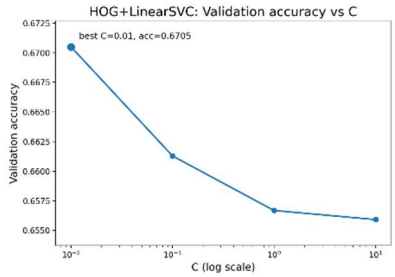

Figure 1. Validation Accuracy vs Regularization Parameter C for LinearSVC trained on HOG features

3.2 Training Dynamics of Deep Learning Models
To better understand the optimization behavior and generalization characteristics of the deep learning models, I examined the training and validation loss curves across 100 epochs for (1) the CNN trained from scratch, (2) ResNet-18 with full fine-tuning, and (3) EfficientNet-B0 with full fine-tuning. For true comparability, all models were trained using identical preprocessing

(RGB images resized to 128×128), batch size 32, Adam optimizer (learning rate 1e-4), and cross-entropy loss for 100 epochs. For each model, the checkpoint with the highest validation accuracy was selected as the final model. The test set was evaluated only once using this best-validation checkpoint. Figure 2 shows training and validation cross-entropy loss across 100 epochs for the three deep learning models.

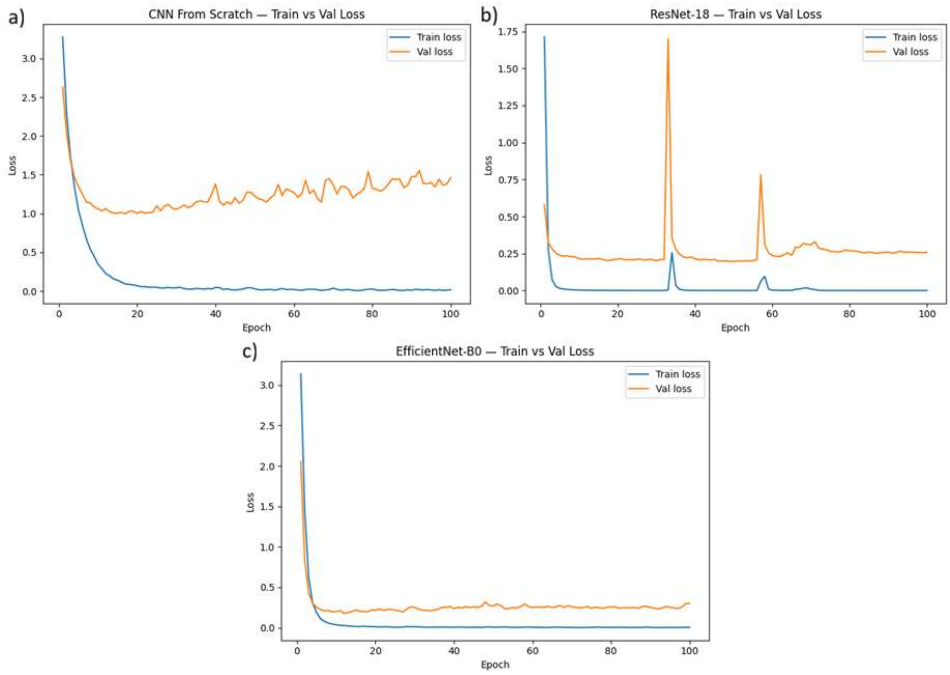

Figure 2. Training and validation cross-entropy loss across 100 epochs for (a) the CNN trained from scratch, (b) the pre-trained ResNet-18, and (c) the pre-trained EfficientNet-B0.

The CNN trained from scratch exhibits overfitting with increased epochs. Training loss rapidly approaches zero, while validation loss reaches a minimum and then steadily increases. The final model was selected at the epoch corresponding to peak validation accuracy (i.e., the checkpoint with highest validation accuracy prior to degradation). Although training loss continued to decrease beyond this point, validation performance declined, indicating memorization of the training data without improved generalization. This widening gap between training and validation loss is characteristic of overfitting in models trained from scratch on moderately sized datasets.

ResNet-18 converges rapidly and maintains low validation loss throughout training. Minor transient spikes are observed but do not persist. The selected checkpoint corresponds to the epoch achieving the highest validation accuracy (0.9508). Compared to the scratch CNN, the validation curve remains substantially more stable, reflecting the benefits of pretrained feature representations and transfer learning.

EfficientNet-B0 demonstrates the lowest and most stable validation loss among all models. The selected checkpoint corresponds to the epoch with highest validation accuracy (0.9555). Despite having fewer parameters than ResNet-18, EfficientNet-B0 exhibits slightly improved validation stability and final performance. It is possible that the deep learning models could have improved with even more epochs of training, but they seem to have stably plateaued within this training paradigm, and regardless, compute and storage limitations prevented me from further training each model.

3.3 Overall Performance

All models were evaluated on the held-out test set using overall accuracy (top-1), per-class accuracy, confusion matrices, precision, recall, and F1 (macro and weighted), and top-5 accuracy.

Table 1 summarizes test performance across all four models. The LinearSVC model trained on HOG features achieved a top-1 accuracy of 0.645 with a macro-F1 of 0.453, reflecting limited generalization and substantial performance imbalance across classes. Training a CNN from scratch improved performance considerably, reaching 0.755 top-1 accuracy and a macro-F1 of 0.627, but still showed a substantial gap relative to pretrained architectures. Both transfer learning models substantially outperformed the scratch and classical baselines. ResNet-18 achieved 0.932 top-1 accuracy with a macro-F1 of 0.896, while EfficientNet-B0 achieved the best overall performance with 0.938 top-1 accuracy and a macro-F1 of 0.905. The consistent improvement in both macro and weighted metrics indicates both higher overall accuracy and better performance across smaller or more difficult classes. Additionally, top-5 accuracy exceeded 0.98 for both pretrained models, demonstrating strong ranking

performance even when the top prediction was incorrect. Figure 3 demonstrates comparisons of top-1 and top-5 accuracies for the four models.

Overall, pretrained models with full fine-tuning substantially outperform both the classical LinearSVC approach and the CNN built from scratch on the Caltech-101 dataset, under the consistent 128×128 RGB preprocessing protocol.

Table 1. Performance of all models. Metrics include accuracy, macro F1, weighted F1, macro precision, macro recall, weighted precision, and weighted recall. All metrics are evaluated on the test set.

Model	**Top-1 Accuracy**	**Top-5 Accuracy**	**Macro precision**	**Macro recall**	**Macro F1**	**Weighted precision**	**Weighted recall**	**Weighted F1**
LinearSVC trained on HOG features	0.645	0.788	0.508	0.453	0.453	0.634	0.645	0.621
CNN trained from scratch	0.755	0.896	0.682	0.621	0.627	0.770	0.755	0.748
ResNet-18 pre-trained	0.932	0.990	0.910	0.895	0.896	0.938	0.932	0.932
EfficientNet-B0 pre-trained	0.938	0.988	0.920	0.902	0.905	0.943	0.938	0.937

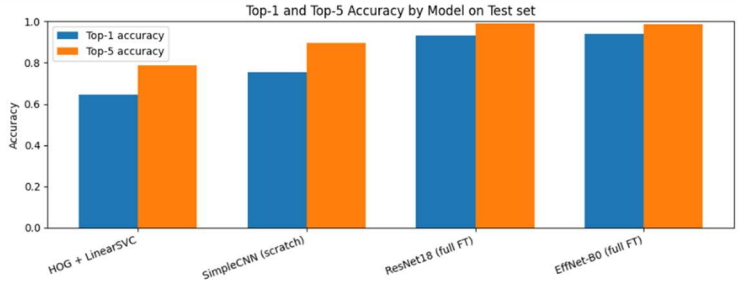

Figure 3. Top-1 and top-5 accuracies of the four models.

3.4 Macro and Weighted Precision, Recall, and F1

Macro metrics weigh each class equally and therefore emphasize performance on rare classes, while weighted metrics account for class frequency, reflecting performance on the dataset distribution as a whole. Comparing macro and weighted scores therefore provides insight into how well each model handles class imbalance. Figure 4 demonstrates macro and weighted F1 for the four models. The LinearSVC trained on HOG features achieved a macro F1 of 0.453 and a weighted F1 of 0.621, revealing a substantial gap between balanced and frequency-weighted performance. While overall accuracy was moderate (0.645), the low macro recall (0.453) indicates significant difficulty in correctly identifying minority classes. This suggests that the classical feature-based approach struggles to generalize across the full diversity of object categories. Training a CNN from scratch improved both macro and weighted metrics, achieving a macro F1 of 0.627 and a weighted F1 of 0.748. The reduction in the gap between macro and weighted metrics relative to the classical baseline indicates improved coverage across classes. However, the macro recall of 0.621 reflects difficulty on less frequent or visually challenging categories. Both pretrained convolutional networks demonstrate strong and well-balanced performance. ResNet-18 achieved a macro F1 of 0.896 and a weighted F1 of 0.932, with macro precision (0.910) and macro recall (0.895) closely aligned, indicating consistent performance across both frequent and rare classes. EfficientNet-B0 achieved the highest macro F1 (0.905) and weighted F1 (0.937), and the strongest macro precision (0.920) and macro recall (0.902). The small gap between macro and weighted metrics for both pretrained models indicates that performance gains are not limited to dominant classes but rather extend broadly across the dataset.

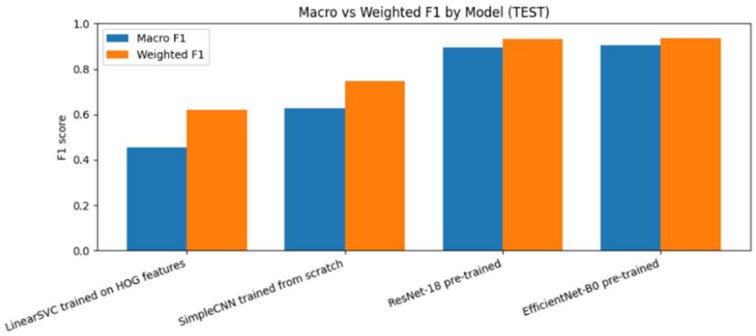

Figure 4. Macro and weighted F1 for all four models

3.5 Confusion Matrices

Figure 5 shows the confusion matrices for the four primary models. The increasing brightness of the diagonal and decreasing bright squares outside of the diagonal in the deep learning models, especially the pre-trained ResNet-18 and EfficientNet-B0, indicates their increased performance compared to the classical machine learning model.

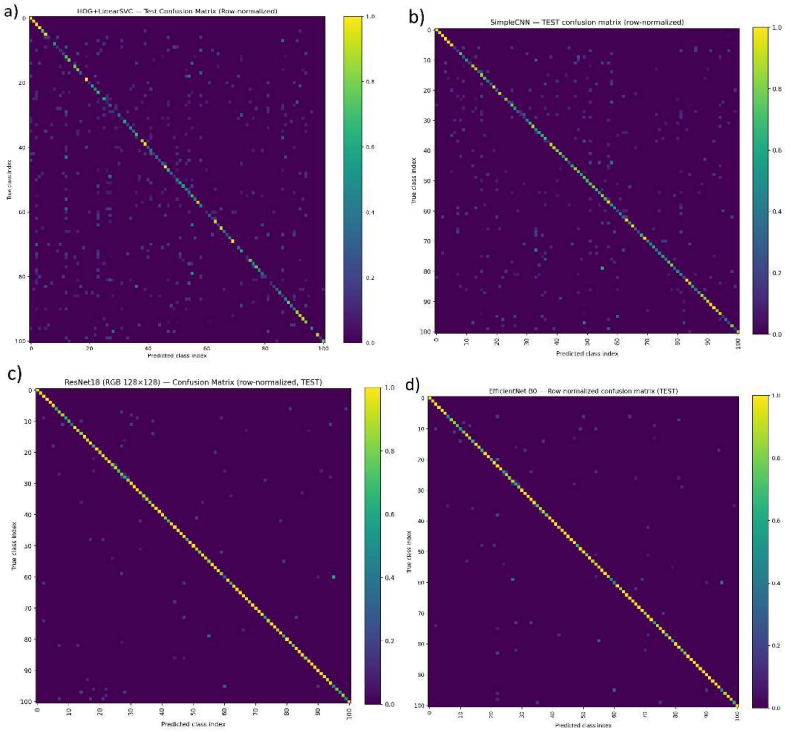

Figure 5. Row-normalized confusion matrices of the four models including (a) The Linear SVC trained on HOG features, (b) the CNN trained from scratch, (c) the pre-trained ResNet-18, and the pre-trained EfficientNet-B0.

3.6 Per-class accuracy and class imbalances

Per-class accuracy (computed as the diagonal of the confusion matrix divided by the number of test samples per class) reveals clear differences between the models. The classical machine learning model shows many classes with very low per-class accuracy and a smaller subset of near-perfect classes. The scratch CNN shifts the distribution upward but still retains a noticeable low-accuracy tail. In contrast, both full fine-tuned pretrained models exhibit per-class accuracy distributions concentrated at high values, indicating robust performance across most classes.

Figure 6 demonstrates class imbalance effects as per-class accuracy compared to class support in the (a) test set and (b) train set (number of images in that class). There is a clear trend that per-class accuracy increases with greater sample size for that class. This indicates that one of the reasons for class imbalances is likely decreased performance on rare classes. This is not necessarily the only reason; it is possible that visually confusing classes also contribute to class imbalances.

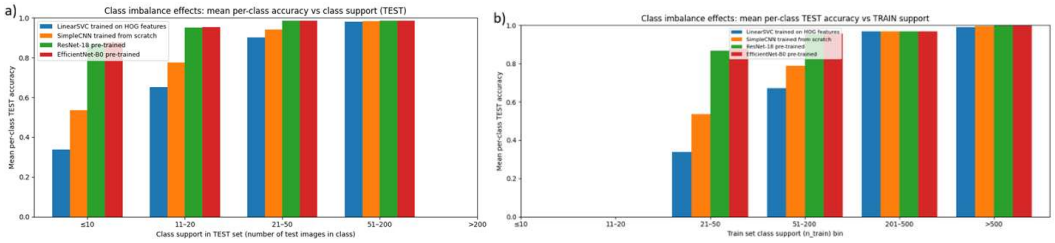

Figure 6. Class imbalance for the four models, demonstrated as (a) per-class accuracy vs class support in the test set and (b) per-class accuracy vs class support in the train set

3.7 Ablation Studies: Effect of Image Size

Reducing image size from 128×128 to 64×64 consistently decreased performance across all models and all metrics (Table 2, Figure 7). The largest absolute drops in performance were observed in the pretrained architectures (ResNet-18 and EfficientNet-B0), suggesting that higher-resolution spatial detail contributes more meaningfully to fine-tuned deep feature representations, which have the capability to more fully leverage the increased data availability. However, performance did decrease in all four models including the CNN trained from scratch and the linear SVC trained on HOG features, demonstrating that larger input images benefit each of these models.

Additionally, each of the models exhibited greater class imbalance effects at the 64x64 image size. The stronger degradation in Macro F1 relative to Weighted F1 across all four models suggests that small or visually ambiguous classes suffer more with the decrease in image size than larger or clearer classes.

Table 2. Performance of all models at 64x64 image size. Metrics include accuracy, macro F1, weighted F1, macro precision, macro recall, weighted precision, and weighted recall. All metrics are evaluated on the test set. The first value is the value of the metric for the 64x64 image size. The second value denoted by a delta is the difference in the metric when image size is decreased, i.e. metric(Δ) = metric(64x64) – metric(128x128).

Model	**Top-1 Accuracy**	**Top-5 Accuracy**	**Macro precision**	**Macro recall**	**Macro F1**	**Weighted precision**	**Weighted recall**	**Weighted F1**
LinearSVC trained on HOG features	0.573 Δ= -0.072	0.742 Δ= -0.046	0.430 Δ= -0.078	0.373 Δ= -0.080	0.381 Δ= -0.072	0.588 Δ= -0.046	0.573 Δ= -0.072	0.568 Δ= -0.053
CNN trained from scratch	0.737 Δ= -0.018	0.894 Δ= -0.002	0.657 Δ= -0.025	0.602 Δ= -0.019	0.602 Δ= -0.025	0.747 Δ= -0.023	0.737 Δ= -0.018	0.725 Δ= -0.023
ResNet-18 pre-trained	0.847 Δ= -0.085	0.942 Δ= -0.048	0.799 Δ= -0.111	0.772 Δ= -0.123	0.772 Δ= -0.124	0.855 Δ= -0.083	0.847 Δ= -0.085	0.842 Δ= -0.090
EfficientNet-B0 pre-trained	0.856 Δ= -0.082	0.952 Δ= -0.036	0.801 Δ= -0.119	0.779 Δ= -0.123	0.778 Δ= -0.127	0.861 Δ= -0.082	0.856 Δ= -0.082	0.851 Δ= -0.086

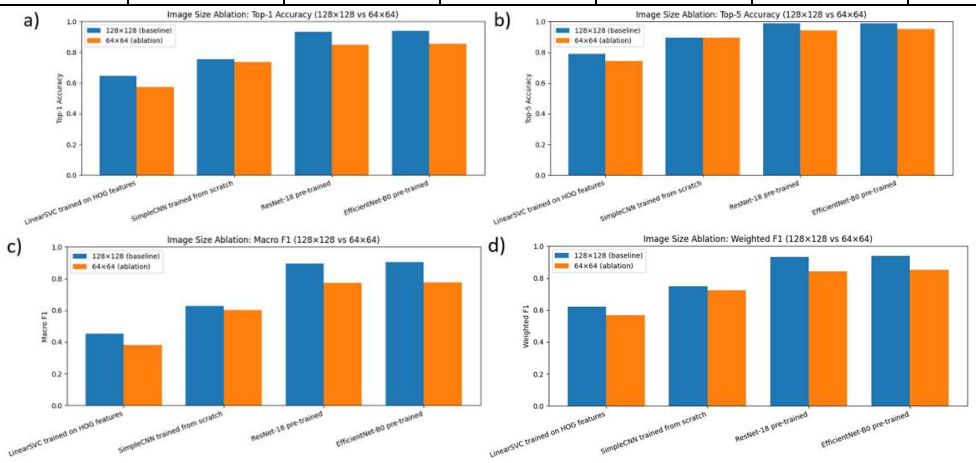

Figure 7. Comparison of (a) top-1 accuracy, (b) top-5 accuracy, (c) macro F1, and (d) weighted F1 with 128x128 vs 64x64 image sizes

3.8 Ablation Studies: Effect of Data Augmentation

The effect of data augmentation generally reduced performance compared with the baseline training (Table 3, Figure 8).

Table 3. Performance of all deep learning models with data augmentation on in training. Metrics are on test set and include accuracy, macro F1, weighted F1, macro precision, macro recall, weighted precision, and weighted recall. The first value is the value of the metric for the augmentation on model. The second value denoted by a delta is the difference in the metric when data augmentation is on, i.e. metric(Δ) = metric(augmentation on) – metric(augmentation off baseline).

Model	**Top-1 Accuracy**	**Top-5 Accuracy**	**Macro precision**	**Macro recall**	**Macro F1**	**Weighted precision**	**Weighted recall**	**Weighted F1**
CNN trained from scratch	0.679 Δ= -0.076	0.862 Δ= -0.034	0.586 Δ= -0.096	0.556 Δ= -0.065	0.550 Δ= -0.077	0.698 Δ= -0.072	0.679 Δ= -0.076	0.674 Δ= -0.074
ResNet-18 pre-trained	0.922 Δ= -0.010	0.988 Δ= -0.002	0.892 Δ= -0.018	0.886 Δ= -0.009	0.882 Δ= -0.014	0.924 Δ= -0.014	0.922 Δ= -0.010	0.919 Δ= -0.013
EfficientNet-B0 pre-trained	0.923 Δ= -0.015	0.992 Δ= 0.004	0.904 Δ= -0.016	0.897 Δ= -0.005	0.891 Δ= -0.014	0.932 Δ= -0.011	0.923 Δ= -0.015	0.921 Δ= -0.016

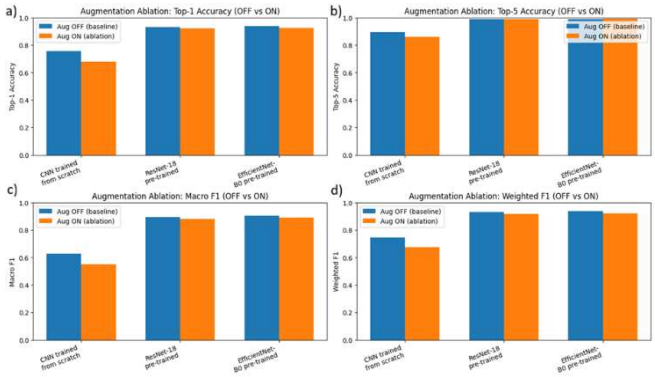

Figure 8. Comparison of (a) top-1 accuracy, (b) top-5 accuracy, (c) macro F1, and (d) weighted F1 with augmentation off vs augmentation on

The SimpleCNN model trained from scratch showed the largest performance decrease when augmentation was introduced. Top-1 accuracy dropped from 0.755 to 0.679, while macro F1 declined from 0.627 to 0.550 and weighted F1 from 0.748 to 0.674. Top-5 accuracy also decreased from 0.896 to 0.862.This reduction suggests that the relatively small convolutional network was sensitive to geometric perturbations, and the added variability introduced by cropping and flipping likely increased optimization difficulty. Because the model was trained entirely from scratch and had limited capacity relative to the pre-trained networks, it may not have been able to fully leverage the augmented samples.

The pre-trained ResNet-18 model exhibited only minor performance changes with augmentation enabled. Top-1 accuracy decreased slightly from 0.932 to 0.922, while macro F1 decreased from 0.896 to 0.882 and weighted F1 from 0.932 to 0.919. Top-5 accuracy decreased very slightly from 0.990 to 0.988. These results indicate that the pre-trained architecture was already able to generalize well without augmentation, and additional transformations slightly decreased performance. EfficientNet-B0 also demonstrated minor changes. Top-1 accuracy decreased from 0.938 to 0.923, while macro F1 decreased from 0.905 to 0.891 and weighted F1 from 0.937 to 0.921. Interestingly, Top-5 accuracy increased slightly from 0.988 to 0.992, suggesting that the model's top-ranked predictions were somewhat less stable, but correct classes remained within the top prediction set.

Across all three architectures, augmentation produced small performance decreases in Top-1 accuracy and F1 metrics, with the largest effect observed for the CNN model trained from scratch. In contrast, the pre-trained models remained relatively stable. These results suggest that, for this dataset and training configuration, the combination of ImageNet pretraining and moderate dataset size (~7,000 training images) provides sufficient generalization. Consequently, additional augmentation may introduce variability that slightly reduces classification performance rather than improving robustness. Figure 8 illustrates these trends across the four key metrics (Top-1 accuracy, Top-5 accuracy, macro F1, and weighted F1). In each panel, the baseline models (augmentation off) consistently perform as well as or better than the augmented variants.

One likely explanation for the observed performance decrease with augmentation is the structure of the Caltech-101 dataset, where objects are typically centered and occupy a large portion of the image frame. Random cropping may therefore frequently remove informative object regions or shift objects away from the framing present in the test set, effectively introducing a distribution mismatch between training and evaluation images. In addition, horizontal flipping may generate unrealistic orientations for certain object categories, further increasing label ambiguity. Because the dataset is already relatively clean and visually consistent, the added variability from augmentation does not substantially improve generalization and instead slightly degrades classification accuracy. It is also possible that the models do not have enough parameters to capture the additional data provided by data augmentation. This effect is most pronounced for the CNN trained from scratch, which lacks the robust feature representations learned through ImageNet pretraining.

4. Discussion

Across the four evaluated approaches, the linear classifier performed the worst, the compact CNN trained from scratch improved substantially, and transfer learning with full fine-tuning (ResNet-18 and EfficientNet-B0) achieved the strongest results. This pattern is consistent with the Caltech-101 dataset, which is moderately sized and visually diverse, so pretrained representations learned from ImageNet provide a strong baseline, while classical features such as HOG capture only limited cues and cannot easily represent the semantic variability across 101 object categories.

A central goal of this project was to compare classical machine learning and deep learning and to examine class imbalance effects. The gap between macro and weighted metrics provides direct evidence of imbalance sensitivity. For the LinearSVC baseline, the weighted F1 score is substantially higher than the macro F1 score, indicating that the model performs disproportionately well on common classes but struggles on rarer ones. In contrast, both pretrained models achieve high and similar macro and weighted F1 scores, implying that performance gains are broadly distributed across classes rather than concentrated. This trend is further supported by the fact that per-class accuracy increases with class support, but the slope is steepest for the classical baseline and is attenuated for the pretrained models, suggesting that transfer learning reduces but does not eliminate the performance inhibition on low-support classes.

The confusion matrices provide a qualitative view of error structure. The classical baseline shows many off-diagonal errors, consistent with limited representational capacity (HOG primarily encodes local gradients and edges). The CNN trained from scratch strengthens the diagonal but still exhibits a noticeable tail of low-performing classes, reflecting the difficulty of learning robust features from limited labeled data. The pretrained models show the strongest diagonal dominance and fewer structured confusions, suggesting that fine-tuned deep features better separate visually similar categories. Even when the top-1 prediction is incorrect, top-5 accuracy remains extremely high for the pretrained models, implying that the correct class is often among a small set of plausible alternatives; this is consistent with errors arising from fine-grained similarity among related object categories rather than complete representational failure.

In these experiments, precision and recall demonstrate different types of performance. Precision measures the accuracy of positive predictions, while recall measures the ability to retrieve positive instances. For the classical LinearSVC baseline, macro precision (0.508) is higher than macro recall (0.453), indicating that when the model predicts a class it is more often correct, but it misses many true instances, especially among rarer classes, which corresponds to the low macro F1 (0.453). The scratch CNN improves both macro precision (0.682) and macro recall (0.621), reducing but not eliminating the class imbalances and increasing macro F1 (0.627). In contrast, the pretrained models achieve high macro precision and recall (ResNet-18: 0.910 vs 0.895 and EfficientNet-B0: 0.920 vs 0.902), meaning they simultaneously reduce false positives and false negatives. Thus, their macro F1 is also high (0.896 and 0.905). Since F1 is the harmonic mean of precision and recall, it summarizes whether a model is doing well on both types of errors. The largest gap between macro and weighted metrics was in the classical model (macro F1 0.453 vs weighted F1 0.621) which supports that its errors are most significant in certain classes, whereas the pretrained models have small gaps indicating more uniform performance across the dataset.

The ablation study on image size demonstrates that higher image size improves performance for all model families. Reducing input from 128×128 to 64×64 decreases overall accuracy and F1 scores across all models, with especially large declines for the pretrained architectures. A likely explanation is that higher resolution preserves fine details that CNN backbones are able to leverage more effectively. Moreover, the stronger degradation in macro F1 relative to weighted F1 at 64×64 indicates that performance on rare or visually ambiguous classes is disproportionately harmed by loss of spatial detail. This result highlights that preprocessing with low image size can have large effects on performance.

In the augmentation ablation, adding random crop/flip transformations slightly reduced performance of the three deep learning models, with the largest drop observed for the CNN trained from scratch. This likely reflects a mismatch between the augmentation and the structure of Caltech-101 images, where objects are often centered and occupy a large fraction of the frame. Augmentation can remove informative object regions and introduce noise. For pretrained models, the performance change is small, suggesting that pretrained representations provide strong generalization and augmentation does not help under the current training procedure. Another potential explanation is that augmentation may require retuning optimization hyperparameters (e.g., longer training or different learning rate), so without retuning, augmentation can increase optimization difficulty without providing a net performance gain.

A limitation of this study is that not all model families were explored (e.g., ViT), largely due to compute and storage constraints. Future work could explore other models, other ablations such as optimizer choice, and and additional data augmentation methods (e.g., weaker crops, color jitter, or automated augmentation).

Overall, transfer learning performs best out of the models tested, with the EfficientNet-B0 achieving the best performance. Pretrained models deliver large gains in both overall and class-balanced performance, while classical methods remain limited. Ablations further show that higher image size improves performance and data augmentation does not always lead to performance improvements.

5. Conclusion

In this project, I compared one classical machine learning baseline (LinearSVC trained on HOG features) and three deep learning approaches (SimpleCNN trained from scratch, pre-trained ResNet-18, and pre-trained EfficientNet-B0) on Caltech-101 using a stratified 70/15/15 split. Overall, transfer learning with full fine-tuning performed best. EfficientNet-B0 achieved the highest test performance (top-1 accuracy 0.938; macro-F1 0.905), closely followed by ResNet-18, while the CNN trained from scratch and LinearSVC trained from HOG features had significantly worse performance. The gap between macro and weighted metrics was largest for the classical baseline, consistent with stronger sensitivity to class imbalance, whereas pretrained models achieved both high overall accuracy and strong class-balanced performance. Two ablation studies demonstrated that reducing image resolution from 128×128 to 64×64 consistently harms performance and exacerbates imbalance effects, and that the tested random crop/flip augmentation slightly reduces performance under this training configuration. These results indicate that pretrained CNNs with full fine-tuning are the most reliable choice for Caltech-101 under the constraints tested, and that image size is important for training efficacy and data augmentation does not always lead to performance improvements.

6. Code Availability

All code can be found at https://github.com/victoria-alkin/SHBT261-class-caltech101-image-classification

References

Li, F. F., et al. (2022). "Caltech 101." CaltechDATA. doi:10.22002/D1.20086